# An analysis on route planning algorithms

Yousif Abbas
*Universiteit Hasselt*
Hasselt, Belgium
yousif.abbas@student.uhasselt.be

***Abstract*—Route planning algorithms are fundamental for many applications, such as the GPS or communication networks. This study benchmarked Dijkstra, A* and their bidirectional variants, each tested against three priority queue architectures: the Binary Min-Heap, the Fibonacci Heap and the Pairing Heap. Experiments were conducted on real-world road networks and synthetically generated graphs. Although the Fibonacci Heap, Pairing Heap and bidirectional searches offer a superior asymptotic complexity in theory, their practical performance is inferior due to constant-factor overheads. A* paired with a Binary Min-Heap proves to be the most effective combination for sparse road networks.**

## I. Introduction

One of the fundamental problems in algorithmic graph theory is the Single Source Shortest Path-problem (SSSP). It serves as a computational backbone for a vast variety of modern technologies, ranging from Global Positioning Systems (GPS) to the routing of data packets through global telecommunications networks and the optimization of supply chain logistics. The ability to rapidly identify the most cost-effective path between two points is essential. As these networks grow in complexity, often comprising millions of vertices and edges, the demand for algorithms that minimize both computational time and memory overhead becomes more and more important.

While Dijkstra's algorithm has served as the standard solution for SSSP on non-negative weighted graphs, the emergence of techniques such as goal-directed heuristics and bidirectional searches have been proven to reduce the search space dramatically [1], [2]. However, the performance of these algorithms is also heavily influenced by the underlying priority queue architecture used to manage the "frontier" of explored nodes. Theoretical breakthroughs, such as the Fibonacci heap, have improved the asymptotic complexity of the *decrease-key* operation, giving Dijkstra's algorithm an improved time complexity.

This study aims to bridge the gap between theoretical complexity and empirical performance. By benchmarking Dijkstra's algorithm, the A* algorithm and their bidirectional variants against the three distinct heap architectures, this study evaluates how different implementation strategies handle large-scale geographic- and random data. Utilizing road network datasets from the 9th DIMACS Implementation Challenge, this study analyses the trade-offs between search space efficiency and execution time efficiency.

## II. Methods

### A. Algorithms

The algorithms this study focusses on are Dijkstra's algorithm and the A* algorithm, along with their bidirectional variants.

*a) Dijkstra's Algorithm:* Dijkstra's algorithm makes use of a priority queue that stores pairs $(v, d[v])$, where $v \in V$ and $d[v]$ represents the tentative distance from the source. The process initializes by setting $d[s] = 0$ for the source node and $d[v] = \infty$ for all other vertices. At each iteration, the algorithm extracts the vertex with the minimum tentative distance in the queue. It then performs edge relaxation on all outgoing edges $(v, u)$, with $v$ being the currently extracted vertex and $u$ a neighbour of $v$; if $d[v] + w(v, u) < d[u]$, the priority queue is updated with the new path cost. This greedy approach ensures that once a node is settled, the shortest path is finalized, provided all edge weights are non-negative [3].

*b) A* Algorithm:* The A* algorithm extends Dijkstra by incorporating a heuristic function, $h(n)$, to bias the search toward the target node $t$. To ensure the algorithm is optimal, an admissible heuristic is employed that never overestimates the true cost to the goal [1]. Nodes in the priority queue are ordered by the evaluation function:

$$f(n) = g(n) + h(n) \tag{1}$$

where $g(n)$ is the cost from the source to node $n$.

Three heuristic candidates were considered:

- **Euclidean distance:** The straight-line distance, calculated as $\sqrt{\Delta x^2 + \Delta y^2}$ [1].
- **Manhattan distance:** The sum of absolute differences, $|\Delta x| + |\Delta y|$.
- **Haversine distance:** Accounting for the Earth's curvature, essential for large-scale geographic road networks.

Manhattan distance is inadmissible on road networks, as diagonal roads can be shorter than the sum of the axes. Haversine is the most geographically accurate. The formula of the haversine distance is:

$$A = \sin^2(\frac{\phi_2 - \phi_1}{2}) + \cos(\phi_1) * \cos(\phi_2) * \sin^2(\frac{\omega_2 - \omega_1}{2})$$
$$d = 2 * r * \arcsin\sqrt{A}$$

with $\phi$ and $\omega$ being respectively the latitude and longitude of each point in radian form.

Its use of trigonometric functions introduces significant per-node computational overhead. Therefore, Euclidean distance was selected as it provides a tight, admissible lower bound on travel cost while remaining computationally inexpensive.

*c) Bidirectional Variants:* Bidirectional variants execute two concurrent searches: a forward search starting from source $s$ and a backward search from target $t$ on the transposed graph $G^T$ [2]. These algorithms reduce the search space complexity exponentially. If a graph has an average degree of $b$, then a unidirectional search will branch to $b$ more nodes each iteration. This means that at the beginning the search branches to the $b$ neighbouring nodes of the source. Then for each neighbour it does the same, making the search branch to $b^2$ nodes in the second iteration, and so on. This continues until the target is reached, which will happen in the $d$-th iteration, with $d$ the distance from source to target. The total visited nodes is given by the summation $\sum_{i=0}^{d}(b^i)$. The search space complexity for unidirectional search is thus $O(b^d)$.

During bidirectional search, the two searches will, optimally, meet in the middle. This means that each search will branch only $d/2$ times, making the total visited nodes $2\sum_{i=0}^{d/2}(b^i)$. The search space complexity for bidirectional search is thus $O(b^{d/2})$, an exponential decrease compared to undirectional search.

At each iteration, a frontier must be chosen to expand. Two techniques were considered: comparing the searches by the number of elements in their priority queues, or comparing the searches by the minimum key in their priority queue. Through empirical analysis, it was found that the first technique gives a major improvement in both search space- and execution time efficiency. A meeting node is recorded whenever a vertex settled by one frontier has already been settled by the other. However, search continues until the stopping criterion is met [4]:

$$min\_key_{fwd} + min\_key_{bwd} \geq \mu \tag{2}$$

where $\mu$ is the cost of the best path found so far. This will be clarified further in the following example.

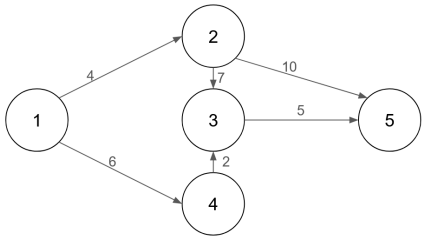


Fig. 1: A random graph to show the working of a bidirectional algorithm. Note that the backward search works on the transposed graph in which all edges point in the opposite direction.

**Example bidirectional Dijkstra**: In this example, the workings of bidirectional Dijkstra will be explained. This example uses the graph shown in Figure 1. In the figures of this example, some cells and nodes will be marked green (forward search) or blue (backward search), this indicates whether it has been visited and finalized by the forward and backward search.

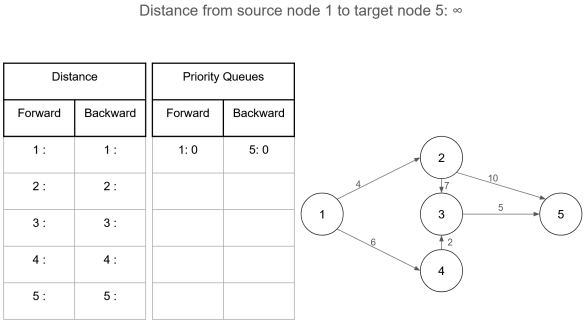


Fig. 2: Before loop

Before the loop starts, the algorithm will place the source- and target node with distance 0 in the forward- and backward priority queue respectively (see Figure 2). The current known distance from the source to the target is set to $\mu = \infty$ as no path is yet known. At each iteration, the algorithm will choose the search with the least amount of elements in their priority queue to expand. In the case of a tie, which is always the case at the start, the forward search will be expanded.

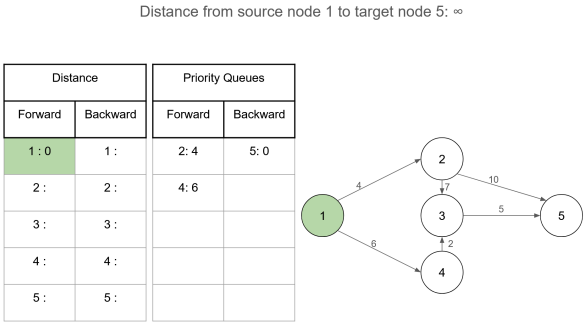


Fig. 3: First iteration

**First iteration**: Once a search has been chosen, the node with the smallest key in the priority queue will be extracted. In this example, this is node 1. The neighbouring nodes will be inserted or updated in the priority queue. In Figure 3, nodes 2 and 4 are inserted with keys 4 and 6 respectively. The distance of the extracted node will also be finalized and the node will be marked as ”visited” as seen in the left table in Figure 3.

Distance from source node 1 to target node 5: ∞

| Distance | | Priority Queues | |
|---|---|---|---|
| Forward | Backward | Forward | Backward |
| 1 : 0 | 1 : | 2: 4 | 3: 5 |
| 2 : | 2 : | 4: 6 | 2: 10 |
| 3 : | 3 : | | |
| 4 : | 4 : | | |
| 5 : | 5 : 0 | | |

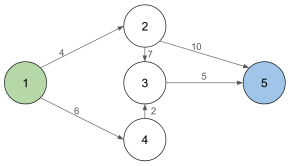


Fig. 4: Second iteration

**Second iteration**: In the second iteration, the backward search has fewer nodes in its priority queue, so it will be expanded next. Node 5, the target, is extracted from the priority queue and its neighbours are inserted into the priority queue. Note that the backward search works on the transposed graph, this is a graph in which all edges point in the opposite direction. This makes node 3 and node 2 accessible to node 5 in the backward search. These nodes are inserted into the priority queue with keys 5 and 10 respectively as shown in Figure 4.

Distance from source node 1 to target node 5: ∞ ⇒ 14

| Distance | | Priority Queues | |
|---|---|---|---|
| Forward | Backward | Forward | Backward |
| 1 : 0 | 1 : | 4: 6 | 3: 5 |
| 2 : 4 | 2 : | 3: 11 | 2: 10 |
| 3 : | 3 : | 5: 14 | |
| 4 : | 4 : | | |
| 5 : | 5 : 0 | | |

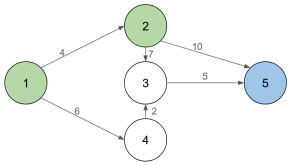


Fig. 5: Third iteration

**Third iteration**: The size of the priority queues is tied, so in this iteration the forward search will be expanded. Node 2 is extracted and finalized with distance 4 as it has the smallest key. Node 2 has neighbours node 3 and 5, since node 5 has already been finalized by the backward search, it means that a path has been found from source to target. The neighbours are inserted with keys 11 and 14 respectively, and $\mu$ is updated to be $dist_{fwd}[2] + w(2,5) + dist_{bwd}[5] = 4 + 10 + 0 = 14$. Although a path has been found, this may not be the shortest path. The sum of the smallest keys in the forward and backward priority queues is currently $6 + 5 = 11$ as shown in the right table in Figure 5. It is smaller than the distance of the path that was just found, so there may be a path that is shorter. Any undiscovered path must pass through a node that is not finalized in both the forward and backward search, so its length cannot be shorter than the sum of the minimum distances remaining in the priority queues. This termination condition only works for graphs with no negative edge weights. A path may still be found with length $l$ where $11 \leq l < \mu$, so the algorithm continues.

Distance from source node 1 to target node 5: 14

| Distance | | Priority Queues | |
|---|---|---|---|
| Forward | Backward | Forward | Backward |
| 1 : 0 | 1 : | 4: 6 | 4: 7 |
| 2 : 4 | 2 : | 3: 11 | 2: 10 |
| 3 : | 3 : 5 | 5: 14 | |
| 4 : | 4 : | | |
| 5 : | 5 : 0 | | |

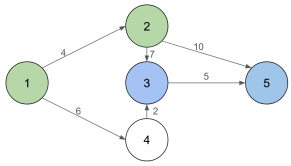


Fig. 6: Fourth iteration

**Fourth iteration**: The backward search is expanded this iteration. Node 3 is extracted and finalized with distance 5. Node 3 has neighbours node 2 and 4 in the backward search. Node 2 is already in the priority queue, so it will have to be updated. However, the distance to node 2 through node 3 is 17, while the distance in the queue is 10, so node 2 will not be updated in the queue. Node 2 is finalized in the forward queue, so another path is found from the source to the target through node 3. However, Figure 6 shows that this path has a length of $dist_{fwd}[2] + w(2,3) + dist_{bwd}[3] = 4 + 7 + 5 = 16$, which is greater than $\mu$, so it will not be updated. Node 4 is also inserted into the priority queue with distance 7. The sum of the smallest keys in both searches is $6 + 7 = 13$, still smaller than $\mu$ so the algorithm continues.

Distance from source node 1 to target node 5: 14 ⇒ 13

| Distance | | Priority Queues | |
|---|---|---|---|
| Forward | Backward | Forward | Backward |
| 1 : 0 | 1 : | 4: 6 | 2: 10 |
| 2 : 4 | 2 : | 3: 11 | 1: 13 |
| 3 : | 3 : 5 | 5: 14 | |
| 4 : | 4 : 7 | | |
| 5 : | 5 : 0 | | |

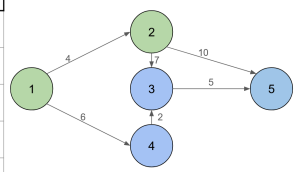


Fig. 7: Fifth iteration

**Fifth iteration**: The backward search is expanded again. Node 4 is extracted and finalized with distance 7. Node 4 only has neighbour node 1, which is a node finalized by the forward search. In Figure 7 a new path is shown with length $dist_{fwd}[1] + w(1,4) + dist_{bwd}[4] = 0 + 6 + 7 = 13$, a length smaller than $\mu$, so $\mu$ is updated to $\mu = 13$. Node 1 is inserted into the priority queue with distance 13. The sum of the smallest keys is now $6 + 10 = 16$, a value greater than $\mu$, so the algorithm terminates.

The bidirectional variant of A* has more complex calculations for the values stored in the priority queue. Instead of using Equation 1, this algorithm uses [4]:

$$f(n) = g(n) + p(n) \tag{3}$$

In which $p(n)$ is a new function defined as [4]:

$$\begin{aligned} p_{fwd}(n) &= \frac{h_{fwd}(n) - h_{bwd}(n)}{2} + \frac{h_{bwd}(t)}{2} \\ p_{bwd}(n) &= \frac{h_{bwd}(n) - h_{fwd}(n)}{2} + \frac{h_{fwd}(s)}{2} \end{aligned} \tag{4}$$

As is explained in *Efficient Point-To-Point Shortest Path Algorithms* [4]: this equation is necessary because two arbitrary functions $h_{fwd}$ and $h_{bwd}$ are not consistent. Their average, however, is both feasible and consistent. To make the algorithm more intuitive, a constant factor of $\frac{h_{bwd}(t)}{2}$ or $\frac{h_{fwd}(s)}{2}$ is added, keeping the functions feasible. Although $p$ gives worse bounds than $h$, it is still worth it in practice due to its consistency.

### B. Priority Queue Architectures

Three heap architectures were implemented and evaluated. Table I summarises their theoretical operation complexities.

| Data Structure | Insert | Decrease-Key | Extract-Min |
|---|---|---|---|
| Binary Min-Heap | $O(\log n)$ | $O(\log n)$ | $O(\log n)$ |
| Fibonacci Heap | $O(1)$ | $O(1)^*$ | $O(\log n)^*$ |
| Pairing Heap | $O(1)$ | $O(\log \log n)^{**}$ | $O(\log n)^*$ |

*Amortized. **Best proven bound; conjectured to be $O(1)$ amortized.

TABLE I: Theoretical operation complexities of the three heap architectures.

*a) Binary Min-Heap:* The Binary Min-Heap serves as the baseline architecture. It is implemented as a complete binary tree using a contiguous array to ensure cache locality. In the context of the SSSP-problem, the implementation requires two primary components [5]:

- **Element Array:** A list of tuples $(v, d[v])$ maintained in heap order.
- **Position Array:** An auxiliary array that tracks the current index of each vertex $v$ within the heap. This is essential for the decrease-key operation, allowing O(1) lookup before the $O(\log(n))$ "bubble-up" process.

Although the Binary Min-Heap is less complex than pointer-based heaps such as the Fibonacci heap and the Pairing heap, it provides a consistent $O(log(n))$ time complexity for the *extract-min*, *insert* and *decrease-key* operations.

*b) Fibonacci Heap:* The Fibonacci Heap was selected for this study due to its theoretical optimization of Dijkstra's algorithm. The heap's name comes from its correlation to the Fibonacci sequence: if a node has degree $n$, its subtree will have at least $F_{n+2}$ nodes to maintain the heap structure [6]. This property binds the maximum degree of any node to $\log_\phi(n)$. By providing an $O(1)$ amortized time for the *insert* and *decrease-key* operations, while maintaining an $O(\log(n))$ amortized cost for *extract-min* [6], it reduces the time complexity of Dijkstra from $O((V + E)\log(V))$ to $O(E + V\log(V))$.

Amortized analysis calculates the worst case time complexity of a sequence of operations rather than focusing on individual executions. It accounts for the fact that while the consolidation during an *extract-min* is costly, it will only happen after many cheap *insert* and *decrease-key* executions have paved the way. If a sequence of $n$ operations have a total worst case time of $O(n)$, then the amortized cost per operation is bounded by:

$$A(n) = \frac{O(n)}{n} \tag{5}$$

This ensures that the total cost of $n$ operations is strictly bounded, effectively distributing the occasional expensive operations across the entire sequence.

Unlike the rigid structure of a Binary Min-Heap, the Fibonacci Heap is a collection of heap-ordered trees. It maintains a single minimum pointer to the root with the smallest key. The efficiency of this structure stems from its "lazy" philosophy: work is deferred until an *extract-min* is performed.

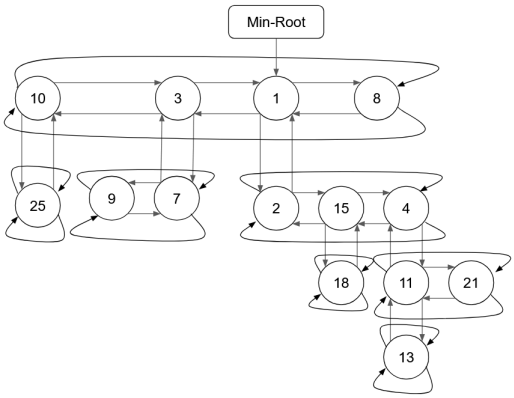


Fig. 8: The structure of a Fibonacci Heap.

The complexity of this heap is shifted to the node level, where each node must maintain the following attributes [6]:

- **Key and Value:** Representing the tentative distance $d[v]$ and the vertex label $v$.
- **Pointers:** A circular doubly-linked list structure as shown in Figure 8, with left and right pointers, to manage siblings at the root or within child lists, along with parent and child pointers for vertical navigation.
- **Cascading Cut Mark:** A boolean value used to track whether a node has lost a child since it was last made a child of another node. This is necessary to ensure the trees remain balanced enough to preserve logarithmic bounds.
- **Degree:** An integer to track the number of children a node has, used during the consolidation phase of extraction.

**Heap Algorithms**:

***insert***: The new node is placed directly into the root list, inserted between the minimum root and the node to its right.

Figure 9 shows what happens to the heap in Figure 8 if an $insert(5)$ is performed.

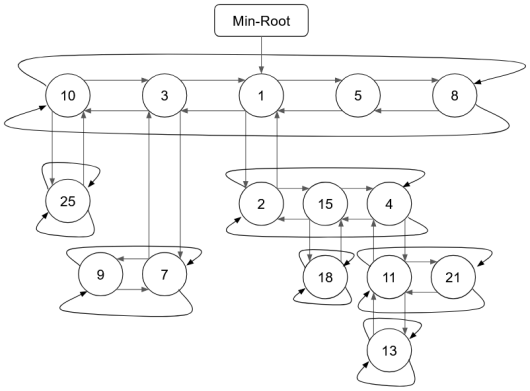


Fig. 9: Fibonacci heap after an insert operation.

***decrease-key***: When node $x$ is decreased to a new key $k$, one of two situations may arise:

- **No violation**: The simplest case in which node $x$ is a root, or the new key $k$ is greater or equal to the key of node $x$'s parent's key. In this situation, nothing is done.
- **Violation**: If $k$ is smaller than node $x$'s parent's key, then node $x$ is *cut* from its parent. The parent's degree is decremented, node $x$'s mark is cleared, and it is inserted into the root list. The minimum root is updated if $k$ is smaller than the minimum root's key. After a cut, the parent is also examined. If the parent is a root, nothing happens. If the parent is not a root and unmarked, it is set to marked. However, if it is not a root and it is marked, the parent will be cut from its parent and inserted into the root list. The same check is done recursively up the tree. This chain reaction is called the *cascading cut*.

The heap in Figure 10 shows what happens if a $decrease\_key(node : 18,\ new\_key : 6)$ is called on the heap in Figure 9. Assume that node 15 was already marked to show the cut operation.

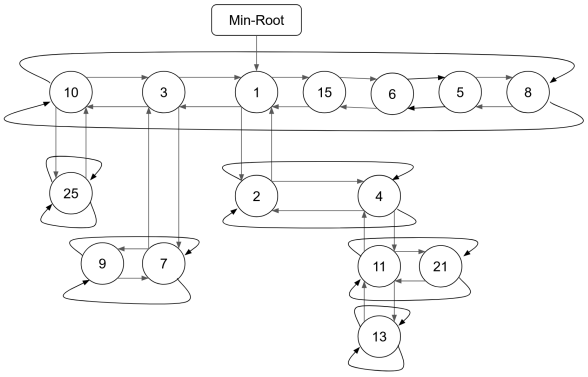


Fig. 10: Fibonacci heap after a decrease-key operation. The heap structure is temporarily lost, but this is intentional and will be corrected during the next extract-min. This "lazy" operation is the key to the heap's efficiency.

***extract-min***: This is the most complex operation and it's where all deferred work is resolved. The minimum root is removed, its children are inserted into the root list and the right and left sibling of the minimum root are linked. The minimum root pointer is updated to be the root with the smallest key and consolidation starts. During consolidation, the algorithm traverses through the root list starting at the minimum root while maintaining a *degree table*, an array indexed by degree, in which each entry holds the root of the unique tree with that degree. When two roots of equal degree are encountered, the one with the larger key is made a child of the other and the entry in the degree table is updated. Traversal continues until every root holds a unique degree. In *Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms* [6] it is proven that the maximum degree of any node in a heap of $n$ nodes is bounded by $\log_\phi(n)$, this formula can be used to efficiently allocate the degree table with minimal unnecessary overhead.

Figure 11 shows the effect of an $extract\_min()$ operation if it is performed on the heap in Figure 10.

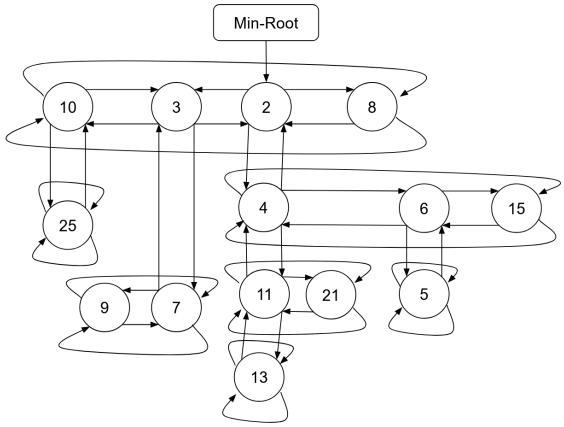


Fig. 11: Fibonacci heap after an extract-min operation. Node 1 is removed and its children (nodes 2 and 4) are inserted into the root list. Consolidation then links roots of equal degree to each other to retain the Fibonacci heap structure.

*c) Pairing Heap:* The Pairing Heap was included in this study due to its reputation as a high-performance, self-adjusting alternative to the Fibonacci Heap. It is often preferred due to its lower constant-factor overhead and superior cache efficiency. The Pairing Heap offers the same $O(1)$ amortized *insert* and $O(\log n)$ amortized *extract-min* as the Fibonacci Heap, while its *decrease-key* has a best proven amortized bound of $O(\log \log n)$, though it is widely conjectured to be $O(1)$ in practice [7], [8]

Its structure is significantly simpler than the Fibonacci Heap as there is no mark bit and no degree field. The heap is a single multi-way heap-ordered tree represented using a left-child, right-sibling pointer scheme. To support the *decrease-key* operation, the left-most child's left pointer will also act as a parent-pointer. The origin of the heap's name lies in the

*pairing* step during *extract-min*: after the root is removed, its children form a forest. A first left-to-right pass merges these trees in consecutive pairs. A second right-to-left pass then merges the resulting trees into a single heap-ordered tree, restoring the structure [7].

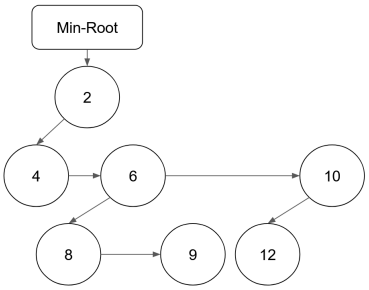


Fig. 12: The structure of a Pairing Heap.

Each node in the Pairing Heap requires fewer attributes than the Fibonacci Heap:

- **Key and Value:** Representing the tentative distance $d[v]$ and the vertex label $v$.
- **Child Pointer:** Pointer to the left-most child of the node.
- **Sibling Pointers:** A doubly-linked list used to connect siblings. The **Prev** pointer is used as a parent pointer for the leftmost child to allow for $O(1)$ access during the *decrease-key* operation.

**Heap Algorithms**:

***insert***: A new single-node tree is created and melded with the existing heap. Melding compares the two minimum roots. The root with the larger key is appended as the left-most child of the root with the smaller key. Figure 13 shows how a new node 13 is inserted into the heap in Figure 12.

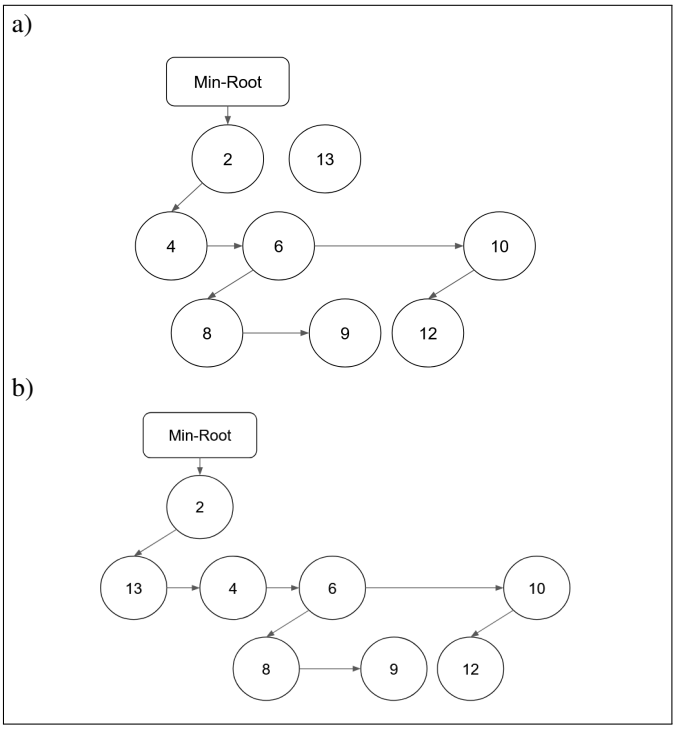


Fig. 13: Insert operation on a Pairing Heap. a) The new node is inserted as a separate single-node tree. b) The new tree is melded with the existing tree.

***decrease-key***: The key of node $x$ is updated. If $x$ is not the root, it is cut from its parent and melded back into the heap as in an *insert*.

Figure 14 shows the effect of a $decrease\_key(node : 12,\ new\_key : 3)$ call on Figure 13.

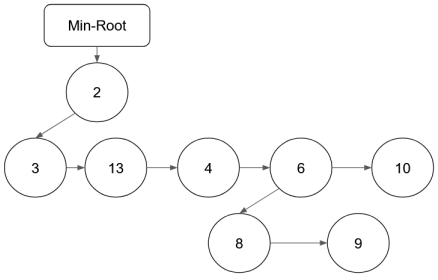


Fig. 14: Node 12 becomes 3 and is cut from its parent. It is then inserted into the heap.

***extract-min***: The minimum root is removed, leaving its children as a forest of separate trees. A left-to-right pass pairs consecutive trees by melding each pair. A right-to-left pass then melds the resulting pairs into a single heap.

An $extract\_min()$ on the heap in Figure 14 proceeds as is shown in Figure 15.

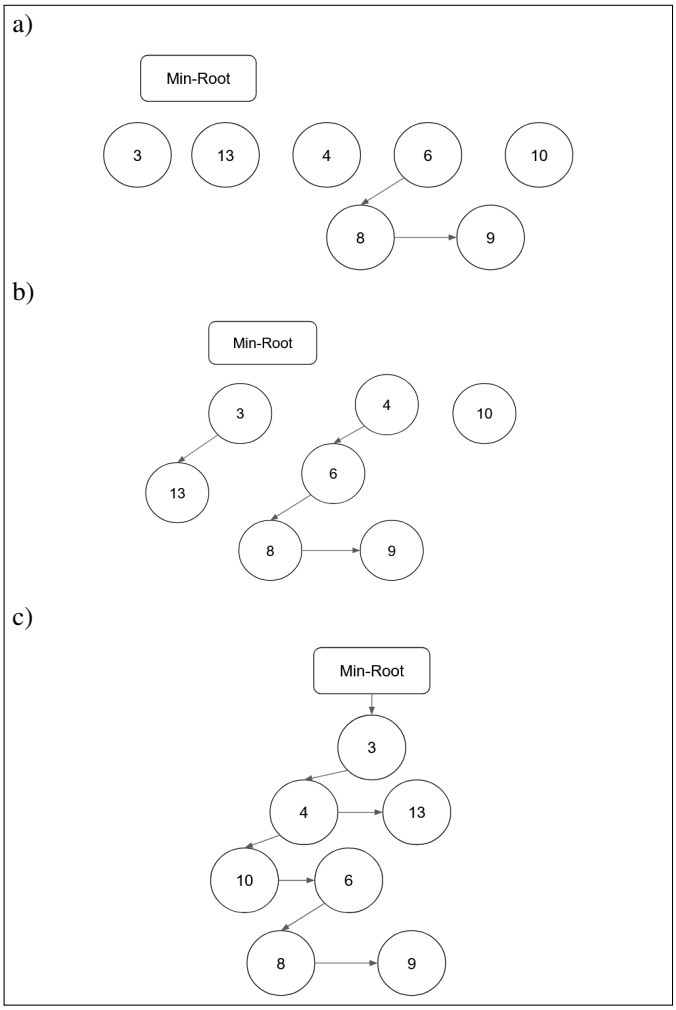


Fig. 15: Extract-min operation on a Pairing Heap. a) Children of extracted root are left as separate trees. b) The trees are paired left to right. c) The resulting pairs are melded from right to left.

**Comparing the Fibonacci Heap and Pairing Heap**: Table I summarises how the two heaps compare theoretically. The key difference lies in the *decrease-key*. The Fibonacci Heap achieves a proven $O(1)$ amortized time through its cascading cut mechanism, whereas the Pairing Heap's equivalent bound is still an open conjecture [8]. However, this theoretical edge comes at a cost. The Fibonacci Heap requires four pointers, a mark bit and a degree field per node, while the Pairing Heap needs only three pointers and no auxiliary fields. This causes the Fibonacci Heap to still perform inferiorly to the Pairing Heap in practice.

### C. Graphs

The performance of SSSP algorithms is significantly influenced by the underlying graph data structure. In this study, an Adjacency List representation will be utilised to model the networks. While an Adjacency Matrix provides $O(1)$ edge lookups, its space complexity of $O(|V|^2)$ is not ideal for large-scale networks like the DIMACS datasets which can reach up to 10 million vertices [9].

By contrast, the adjacency list requires only $O(|V| + |E|)$ space. Given that road networks are inherently sparse, the adjacency list ensures that the graph fits within the system's primary memory while maintaining efficient $O(degree(v))$ iteration over neighbouring vertices during the relaxation phase.

While adjacency lists are best implemented using static arrays for the vertices and edges, it was decided that a dynamic approach using a linked list would be better for synthetically generated random graphs.

### D. Experimental Setup

*a) Benchmark Datasets:* To evaluate the algorithms, standard road network benchmarks from the 9th DIMACS Implementation Challenge were used. These graphs represent real-world topologies where vertices correspond to geographic coordinates and edges represent road segments with travel time weights. Specifically, the following datasets were utilized:

- **USA-road-d.CAL (California):** A large, sparse graph [9].
- **USA-road-d.NY (New York):** A smaller sparse graph [9]
- **Random Graphs:** Generated using a variant of the Watts-Strogatz model for generating random small-world graphs. Used in this study to show the effects of dense graphs on the execution time. [10].

| Graph Name | Nodes ($\|V\|$) | Edges ($\|E\|$) |
|---|---|---|
| California | 1,890,815 | 4,657,742 |
| New York | 264,346 | 733,846 |
| Random Graph | 10,000-$10^8$ | 20,000-$10^9$ |

TABLE II: Properties of the road network graphs used for performance benchmarking.

The Watts-Strogatz on a graph with $N$ nodes and a mean degree of $K$ works as follows [10]:

1) **Construct a ring lattice**: Connect the nodes of a graph with $N$ nodes to its $K$ nearest neighbours by undirected edges.
2) **Rewiring**: A vertex and the edge that connects it to its nearest neighbour in clockwise sense is chosen. With probability $p$, the edge is rewired to connect the chosen vertex with a random uniformly chosen vertex over the entire graph, with duplicate edges being forbidden. If a duplicate exists, the rewiring is skipped. This is repeated by moving clockwise until each vertex is considered. Next, the edge connecting each vertex to its second nearest neighbour is considered, and so on.
3) **Termination**: The algorithm concludes once all edges in the original lattice graph have been considered.

This algorithm creates 'small-world' graphs whenever $0 < p < 1$. Small-world networks are graphs that have a high clustering coefficient and low distances. High clustering implies that two nodes that are connected to the same node,

are probably connected themselves. The low distances on the other hand, also known as the six degrees of separation [10], means that all nodes are six or less connections away from each other.

The variant used in this study changes the rewiring phase. It also allows for a fixed amount of edges $E$ to be given. The variant works as follows:

1) **Construct a ring lattice**: The same way the Watts-Strogatz algorithm does, this variant connects the nodes of a graph with $N$ nodes to its $K$ nearest neighbours.
2) **Shuffle**: This is where the variant differs. At each iteration, the vertices of the graph, which were labelled as $0, 1, ..., N-1$ at first, are shuffled randomly. The shuffled nodes are then once again connected as a lattice, but they are connected to the $K$ nearest neighbours in the shuffled order. Duplicate edges are not allowed. If vertices $i$ and $j$ are already connected, then they will be skipped if they appear again in the lattice construction.
3) **Termination**: The algorithm concludes once $E$ edges have been created.

This variant was made because it was easier to enforce a fixed amount of vertices and edges and it gave the algorithm a noticeable improvement in execution time. However, this makes the graph lose its small-world properties.

*b) Implementation Details:* The algorithms and data structures were implemented in C. The priority queues were built from scratch to ensure that the memory overhead of the Fibonacci and Pairing heaps could be measured accurately against the Binary heap. The visualisations made in Figure 17 were made in python using the *pandas*, *datashader*, *datashader.transfer_functions* and *PIL.Image* libraries. During execution, the search data was saved into a *csv*-file that the python script could read and use to render the search results.

## III. Results

The following sections provide an empirical analysis of the different search algorithms and the data structures that were used. Each algorithm/data structure was tested 3000 times with random source-target pairs to find the averages of all queries. Execution time and search space coverage were used as the measures to evaluate these algorithms. These results are from experiments conducted on the road networks in Table II. The random graphs were not included in the *Search Efficiency* section as A* and Bidirectional A* could not be tested on them. No admissible heuristic exists for graphs with random edge weights as they hold no geographical meaning.

### A. Search Efficiency

| Algorithm | California | | New York | |
|---|---|---|---|---|
| | Time (s) | Visit % | Time (s) | Visit % |
| Dijkstra | 0.182 | 51.54% | 0.024 | 49.09% |
| Bidirectional Dijkstra | 0.208 | 40.30% | 0.020 | 32.50% |
| A* | 0.100 | 14.22% | 0.007 | 12.31% |
| Bidirectional A* | 0.157 | 12.78% | 0.017 | 8.75% |

TABLE III: Average performance metrics across both road networks.

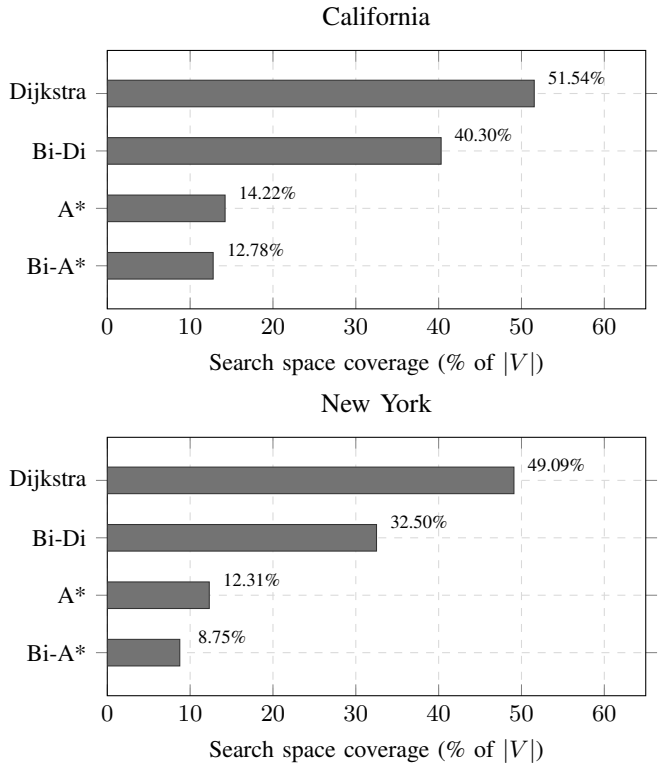


Fig. 16: Search space coverage per algorithm on the California and New York road networks.

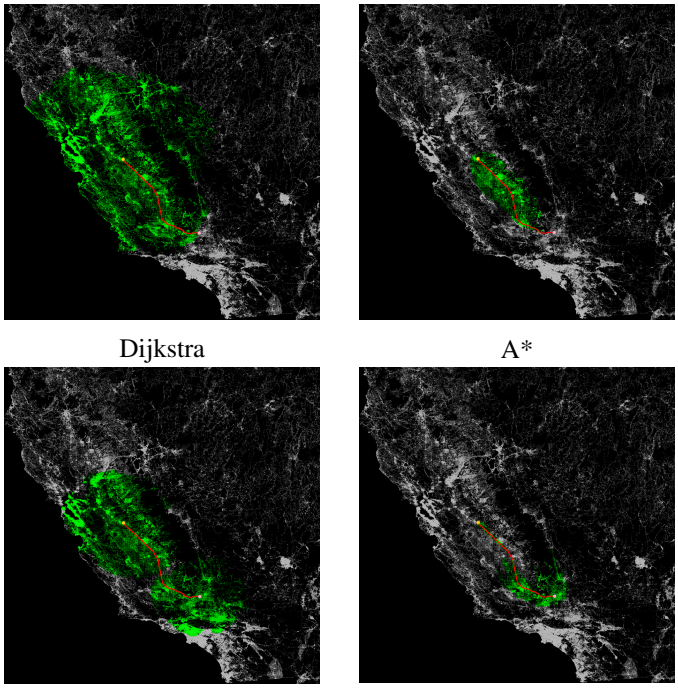


Fig. 17: Nodes visited by each algorithm on the California road network, coloured green (visited), yellow (source) and pink (target).

Looking at the results of the experiments on the California road network, a clear trend was shown on Figure 16 and Figure 17 between the unidirectional and bidirectional variants. While bidirectional search is designed to limit the expansion of the search frontier, the execution time for the bidirectional variants in this specific network actually increased compared to their unidirectional counterparts. Bidirectional Dijkstra was approximately $14\%$ slower than Dijkstra, and Bidirectional A* was $57\%$ slower than A*. This was due to the computational overhead of maintaining two separate priority queues and the complexity of checking stopping criteria at each iteration (see Equation 2).

Bidirectional A* still managed to run faster than Bidirectional Dijkstra despite the heavy computations it must perform at each iteration (see Equation 4). This was due to Bidirectional A* cutting the search space by a significant amount. During the experiments, it visited around $70\%$ fewer nodes relative to Bidirectional Dijkstra, thus outweighing the overhead from the mathematical computations.

On the New York road network, however, a different trend was observed. In this smaller, denser road network, the bidirectional variants showed improvements in both execution time and search space coverage compared to standard Dijkstra as is shown in Figure 16. Bidirectional Dijkstra ran queries $16.6\%$ faster than normal Dijkstra, while reducing the search space coverage by roughly $17\%$, a relative reduction of approximately $33\%$ compared to Dijkstra's coverage. Bidirectional A* achieved the lowest search space coverage of only $8.75\%$, however it fell short in its execution time, performing worse than unidirectional A*.

The A* algorithm consistently showed the best balance between execution time and search space coverage across both networks. It searched only $14.22\%$ and $12.31\%$ of California and New York respectively, presenting a dramatic reduction in visits compared to Dijkstra ($51.54\%$ and $49.09\%$ respectively). This reduction in node expansion translated directly into execution speed. A* was the fastest algorithm in both networks, performing $45\%$ faster than Dijkstra in California and $70\%$ faster in New York.

### *B. Architecture Efficiency*

For the analysis of different data architectures only the execution time was considered as the choice of data structure did not affect the number of nodes visited. These results are based on the execution of normal Dijkstra with each of the three heap implementations on the California road network and a randomly generated graph.

| **Data Structure** | **Avg. Time ( $s$)** |
|---|---|
| Binary Min-Heap | 0.182 |
| Fibonacci Heap | 0.356 |
| Pairing Heap | 0.236 |

TABLE IV: Average execution time of Dijkstra's algorithm under different heap architectures on the California road network.

| **Data Structure** | **Avg. Time ( $s$)** |
|---|---|
| Binary Min-Heap | 0.040 |
| Fibonacci Heap | 0.040 |
| Pairing Heap | 0.041 |

TABLE V: Average execution time of Dijkstra's algorithm under different heap architectures on a randomly generated dense graph ($V = 10^4, E = 10^6$, average degree of $\sim 200$).

The results in Table IV show that there was a clear performance hierarchy among the three heap architectures. The Binary Min-Heap achieved the fastest execution at 0.18 s, followed by the Pairing Heap at 0.24 s ($1.30\times$ slower), with the Fibonacci Heap performing worst at 0.36 s, nearly double the Binary Min-Heap's runtime.

The underperformance of the Fibonacci Heap was particularly notable given its theoretical advantages. While it reduced Dijkstra's time complexity from $O((V + E)\log(V))$ to $O(E + V\log(V))$ by lowering the amortized cost of *insert* and *decrease-key* from $O(\log(n))$ to $O(1)$, these gains were not shown in practice on road networks. This was caused by significant constant-factor overhead introduced by its pointer-heavy node structure: each node needs a left, right, parent and child pointer. This is not cache-friendly and caused per-operation cost that far outweighs the asymptotic advantages on sparse graphs.

The Pairing Heap gave a middle ground between the Fibonacci Heap and the Binary Min-Heap. Its implementation is much simpler than the Fibonacci Heap, using only three pointers instead of four, giving better cache behaviour. Yet it incurs more pointer traversal overhead than the array-based Binary Min-Heap. This explains its intermediate performance.

During the execution of Dijkstra's algorithm, there are: $|V|$ *insert* calls, $|V|$ *extract-min* calls and $|E|$ *decrease-key* calls. In dense graphs where $E \approx V^2$, the algorithm's *decrease-key* calls will increase quadratically, while *extract-min* calls will only increase linearly, making the Fibonacci Heap benefit much more than the Binary Min-Heap due to its improvements to *decrease-key*'s time complexity. Fibonacci Heaps would perform better than Binary Min-Heaps in these situations. However, road networks are inherently sparse, the average vertex degree in the California dataset is approximately $2|E|/|V| \approx 4.9$, meaning $E = O(V)$ rather than $O(V^2)$, that is why these architectures performed worse than the Binary Min-Heap on both road networks. Although the Pairing Heap is thought to have an amortized time complexity of $O(1)$ for its *decrease-key* operation, the best proven bound remains $O(\log\log(n))$ [8]. This means that the Pairing Heap will not benefit as much from dense graphs as the Fibonacci Heap. This is also shown in Table V, as the density of the graph increases, the Fibonacci Heap and Pairing Heap will close the performance gap with the Binary Min-Heap, eventually overtaking it.

## CONCLUSIONS

This study set out to bridge the gap between theoretical complexity of shortest path algorithms and their empirical

performance on large-scale road networks. By benchmarking Dijkstra's algorithm, A* and their bidirectional variants against three heap architectures on the DIMACS real-world road network dataset, two key measures of performance were evaluated: search space efficiency and execution time.

The results demonstrate that informed searches offer the most practical improvement over standard Dijkstra, with A* giving a balance between execution time and search space reduction. By reducing the average node visits by $75\%$ and average execution time by $46\% - 71\%$, A* achieves a better trade-off than either bidirectional variant. Bidirectional search, while effective at pruning the search space, can have increased execution time on larger networks. This suggests that on sparse road networks, the constant factor cost of bidirectional searches outweighs its search-space savings.

The heap architecture experiments reveal an equally important gap between asymptotic theory and practical performance. Despite its theoretical superiority, the Fibonacci Heap nearly doubled execution time relative to the Binary Min-Heap. Its pointer-heavy node structure introduces memory access patterns that are cache-unfriendly. This dominates the runtime on sparse graphs where $E = O(V)$, which is usually the case with road networks (average degree of California and New York is $\sim 4$). The Pairing Heap, while simpler and more cache-efficient than the Fibonacci Heap, still incurs more pointer traversal overhead than the array-based Binary Min-Heap. These findings confirm that on road network topologies, the cache locality of a simple array-based structure outweighs the asymptotic advantages of pointer-based heaps.

Taken together, these results suggest that A* paired with a Binary Min-Heap represents the most effective combination for practical route planning on sparse road networks. The theoretical gains of more complex data structures and bidirectional search strategies are unlikely to show unless the graph topology changes significantly. Specifically, on dense graphs where $E \approx V^2$ and *decrease-key* operations dominate the runtime.

Future work could investigate the performance of techniques built upon the algorithms that were studied here, such as precomputing with landmarks, reaches or shortcuts. The ALT-algorithm (A*, Landmarks and Triangle Inequality) extends the A* algorithm by adding landmarks that pre-process the distance to nearby nodes [11]. Reach-based routing assigns each node a reach value reflecting its importance, allowing nodes with low reach to be pruned early and reducing the effective search space. Shortcuts preprocess the graph by iteratively removing unimportant nodes and inserting shortcut edges to preserve shortest path distances, allowing query times to be reduced by orders of magnitude and making it a practical choice for real-world GPS systems [4].

## Self Reflection

This project challenged several assumptions i had going in. I expected pointer-based heaps and bidirectional search to outperform their simpler counterparts, given their theoretical advantages. Seeing that it is the opposite was a valuable lesson. By implementing these algorithms and data structures in C, it was easier to understand why some algorithms and data structures performed worse in practice, namely, because factors like cache behaviour and pointer overhead can dominate runtime, making the algorithms lose their theoretical advantage.

Although I used real-world graphs from the DIMACS data to try and simulate the performance of the algorithms and data structures in real world scenarios, my implementation still lacks dynamic factors that route-planning applications struggle with. In the future, it might be interesting to explore the incorporation of dynamic edge weights to model real-world factors such as traffic congestion, road closures, or time-of-day variation, which would more closely reflect the conditions faced by practical route planning systems. Another limitation of this study is the lack of statistical analysis. Reporting only averages over 3000 queries, without confidence intervals or variance measures, makes it difficult to assess how stable the results are across different source-target pairs.

## Acknowledgments

I sincerely thank Prof. Dr. Jan Van Den Bussche. His guidance proved invaluable in shaping the direction and quality of this study. He broadened my horizons by showing me new perspectives and encouraging deeper analysis. I also thank Universiteit Hasselt for providing the resources and environment that made this research possible.

## References

[1] P. E. Hart, N. J. Nilsson and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

[2] I. Pohl, B. Meltzer and D. Michie, "Bi-directional Search," *Machine Intelligence*, vol. 6, pp. 127–140, Edinburgh University Press, 1971.

[3] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

[4] A. V. Goldberg, C. Harrelson, H. Kaplan and R. F. Werneck, "Efficient Point-To-Point Shortest Path Algorithms," unpublished, Princeton University, 2006.

[5] J. W. J. Williams, "Algorithm 232 - Heapsort," *Communications of the ACM*, pp. 347–348, 1964.

[6] M. L. Fredman and R. E. Tarjan, "Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms," *Journal of the ACM*, vol. 34, pp. 338–346, 1984.

[7] M. L. Fredman, R. Sedgewick, D. D. Sleator and R. E. Tarjan, "The Pairing Heap: A New Form of Self-Adjusting Heap," *Algorithmica*, vol. 1, pp. 111–129, 1986.

[8] M. L. Fredman, "On the Efficiency of Pairing Heaps and Related Data Structures," *Journal of the ACM*, pp. 473–503, 1999.

[9] DIMACS, "DIMACS 9th Implementation Challenge," 2006. [Online]. Available: https://www.diag.uniroma1.it/~challenge9/download.shtml. [Accessed: May 9, 2026]

[10] D. J. Watts and S. H. Strogatz, "Collective dynamics of 'small-world' networks," *Nature*, vol. 393, pp. 440–442, 1998.

[11] A. V. Goldberg and C. Harrelson "Computing the Shortest Path: A* Search Meets Graph Theory," *Proceedings of the Sixteenth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA '05)*, pp. 156–165, 2005.